

Porównanie udostępnianych na otwartych licencjach rozproszonych silników zapytań pod kątem analiz danych genomicznych

Miłosz Kowalewski

Promotor: dr inż. Marek Wiewiórka

Analiza literatury i źródeł

Uwaga metodologiczna

Poniższe pozycje dzielą się na dwie kategorie: - **Publikacje naukowe** — artykuły w recenzowanych czasopismach i materiałach konferencyjnych - **Źródła techniczne** — repozytoria, dokumentacja, specyfikacje

Repozytoria i dokumentacja techniczna mogą być cytowane w pracy magisterskiej jako źródła techniczne, jednak nie zastępują publikacji naukowych w rozdziale “analiza literatury”. Do zweryfikowania z wymaganiami uczelni.

Publikacje naukowe

Genomika i operacje interwałowe

[1] polars-bio (2025)

Wiewiórka M., Khamutou P., Zbysiński M., Gambin T. *polars-bio — fast, scalable, and out-of-core operations on large genomic interval datasets*. Bioinformatics, 41(12), btaf640, 2025. DOI: 10.1093/bioinformatics/btaf640

Główna biblioteka będąca przedmiotem pracy. Implementuje operacje genomiczne (overlap, merge, nearest, coverage, subtract) w oparciu o Apache DataFusion i Polars. Autorzy wykazują 6–38x przyspieszenie względem bioframe oraz wsparcie dla strumieniowego przetwarzania danych przekraczających rozmiar RAM.

[2] BEDTools (2010)

Quinlan A.R., Hall I.M. *BEDTools: a flexible suite of utilities for comparing genomic features*. Bioinformatics, 26(6), 841–842, 2010. DOI: 10.1093/bioinformatics/btq033

Narzędzie będące de facto standardem dla operacji na interwałach genomicznych. Punkt odniesienia dla polars-bio i większości nowszych bibliotek. Praca ta definiuje zestaw operacji (overlap, merge, subtract itd.), który stanowi cel integracji w niniejszej pracy magisterskiej.

[3] bioframe (2024)

Open2C, Abdennur N., Fudenberg G., Flyamer I.M., et al. *Bioframe: operations on genomic intervals in Pandas dataframes*. Bioinformatics, 40(2), btae088, 2024. DOI: 10.1093/bioinformatics/btae088

Biblioteka Pythonowa do operacji genomicznych oparta na Pandas. Stanowi bezpośredni punkt porównania dla polars-bio (API wzorowane na bioframe). Reprezentuje podejście “Python-first” bez optymalizacji natywnych.

[4] Augmented Interval List — AIList (2019)

Feng J., Ratan A., Sheffield N.C. *Augmented Interval List: a novel data structure for efficient genomic interval search*. Bioinformatics, 35(23), 4907–4911, 2019. DOI: 10.1093/bioinformatics/btz407

Opisuje strukturę danych AIList jako alternatywę dla drzew interwałowych. Istotna dla rozdziału o algorytmach — polars-bio oferuje wybór algorytmu (COITrees, Lapper, IntervalTree), a porównanie ich złożoności obliczeniowej w kontekście rozproszonym jest ważnym elementem analizy.

[5] SparkGA2 (2019)

Mushtaq H., Ahmed N., Al-Ars Z. *SparkGA2: Production-quality memory-efficient Apache Spark based genome analysis framework*. PLOS ONE, 14(12), e0224784, 2019. DOI: 10.1371/journal.pone.0224784

Przykład praktycznego zastosowania Apache Spark w genomice. Pokazuje że rozproszenie obliczeń genomicznych przez Sparka jest wykonalne, ale wymaga istotnych nakładów inżynierskich. Kontekst dla pytania: czy można uniknąć Sparka używając Ballistry.

Silniki zapytań i obliczenia rozproszone

[6] Apache DataFusion — SIGMOD 2024

Lamb A., Shen Y., Heres D., Chakraborty J., Kabak M.O., Hsieh L., Sun C. *Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine*. Proceedings of SIGMOD 2024, Santiago, Chile, 2024. DOI: 10.1145/3626246.3653368

Artykuł opisujący architekturę DataFusion — silnika zapytań będącego fundamentem zarówno polars-bio jak i Ballistry. Kluczowy dla zrozumienia mechanizmu PhysicalOptimizerRule i systemu rozszerzeń UDF/UDTF, który jest centralnym tematem pracy.

[7] Resilient Distributed Datasets — Apache Spark (2012)

Zaharia M., Chowdhury M., Das T., Dave A., et al. *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*. USENIX NSDI 2012, San Jose, CA. Best Paper Award. URL: <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>

Fundamentalna praca opisująca model RDD i architekturę Apache Spark. Kontekst dla rozdziału o silnikach rozproszonych — Spark jest punktem wyjścia do zrozumienia dlaczego alternatywy takie jak Ballista (bez JVM) są rozważane w nowoczesnych systemach.

[8] MapReduce (2004)

Dean J., Ghemawat S. *MapReduce: Simplified Data Processing on Large Clusters*. OSDI’04, San Francisco, CA, 2004. DOI: 10.1145/1327452.1327492

Pierwotny model obliczeń rozproszonych na dużą skalę. Historyczny punkt odniesienia — Spark powstał jako odpowiedź na ograniczenia MapReduce (brak obsługi danych w pamięci). Wyjaśnia ewolucję w kierunku współczesnych silników takich jak DataFusion i Ballista.

[9] Volcano Query Evaluation Model (1994)

Graefe G. *Volcano — An Extensible and Parallel Query Evaluation System*. IEEE Transactions on Knowledge and Data Engineering, 6(1), 120-135, 1994. DOI: 10.1109/69.273032

Klasyczny model wykonania zapytań, na którym opiera się architektura DataFusion (i większości współczesnych silników SQL). Niezbędny do zrozumienia pojęcia PhysicalOptimizerRule i planu wykonania zapytania — kluczowych w kontekście integracji polars-bio z Ballistą.

Źródła techniczne

[10] **polars-bio — repozytorium** > <https://github.com/biodatageeks/polars-bio> > Dostęp: 2025

Kod źródłowy biblioteki. Analiza implementacji PhysicalOptimizerRule (reguły dla overlap/nearest) oraz mechanizmu rejestracji UDF/UDTF w DataFusion.

[11] **Apache DataFusion Ballista — repozytorium** > <https://github.com/apache/datafusion-ballista> > Dokumentacja rozszerzeń: <https://datafusion.apache.org/ballista/user-guide/extending-components.html> > Dostęp: 2025

Rozproszony silnik zapytań wybrany jako cel integracji. Dokumentacja opisuje mechanizm `override_function_registry` i `override_session_builder` umożliwiający rejestrację zewnętrznych funkcji bez modyfikacji kodu źródłowego Ballistry.

[12] **LakeSail — issue #1062 / discussion #2001: Supporting Sail Extensions** > <https://github.com/lakehq/sail/issues/1062> (zamknięty 28.05.2026 jako “completed”) > <https://github.com/lakehq/sail/discussions/2001> (kontynuacja dyskusji, aktywna min. do 28.07.2026) > Dostęp: 2026-08

Dyskusja architektoniczna maintainerów Saila dot. mechanizmu rozszerzeń (SailExtension/FFI dla UDF/UDTF/optimizer rules/plan extensions, bez forka). **Aktualizacja (sierpień 2026):** mechanizm jest wciąż w fazie projektowej, niezaimplementowany — zespół SedonaDB prowadzi równoległą, analogiczną integrację (spatial join $\approx$ genomic interval overlap) na forku [james-willis/sail](#) (branch `sedona-integration`), co stanowi wartościowy precedens/materiał porównawczy. Bez forka Sail oferuje dziś jedynie rejestrację Python UDTF (PR #1519, merged) — to jest w tej pracy traktowane jako docelowy, legalny (bez forka) poziom integracji dla Saila, nie jako powód odrzucenia kandydata.

[13] **ballista_extensions — przykładowa implementacja rozszerzenia** > <https://github.com/milenkovic> > Dostęp: 2025

Projekt demonstracyjny pokazujący jak zaimplementować niestandardowy operator (`sample`) w Ballistce bez forkowania. Punkt wyjścia dla implementacji `DistributedOverlapRule` w ramach niniejszej pracy.

[14] Apache Arrow — specyfikacja formatu kolumnowego > <https://arrow.apache.org/docs/format/Columnar.html> > Dostęp: 2025

Specyfikacja formatu in-memory używanego do wymiany danych między polars-bio, DataFusion i Ballistą. Kluczowa dla rozdziału o architekturze — zero-copy transfer danych między węzłami klastra.

[15] COITrees — repozytorium > <https://github.com/dcjones/coitrees> > Dostęp: 2025

Implementacja struktury danych Cache Oblivious Interval Trees używanej przez polars-bio jako domyślny algorytm dla operacji overlap i nearest. Istotna dla rozdziału o algorytmach — w scenariuszu rozproszonym COITrees działa lokalnie na każdym węźle po partycjonowaniu danych według chromosomu.

[16] datafusion-bio-functions — repozytorium > <https://github.com/biodatageeks/datafusion-bio-functions> > Dostęp: 2026-08

Rodzina cratów Rust (m.in. datafusion-bio-function-ranges) implementująca faktyczny silnik algorytmiczny stojący za polars-bio — eksponuje `overlap(table1, table2)` jako gotową DataFusion table function z wyborem algorytmu (Coitrees, IntervalTree, Lapper, SuperIntervals). Apache-2.0, “designed to be consumed by downstream libraries” — kluczowa zależność dla prototypów w tej pracy (zamiast reimplementacji naiwnego warunku overlap).

[17] SedonaDB × Sail — integracja jako precedens > <https://github.com/james-willis/sail> (branch sedona-integration) > Dyskusja projektowa: <https://github.com/lakehq/sail/discussions/2001> > Dostęp: 2026-08

Równoległy, analogiczny problem badawczy: SedonaDB (silnik danych przestrzennych) integruje spatial join z Sailem — strukturalnie ten sam problem co genomic interval overlap (warunek zasięgu zamiast warunku równości w joinie). Aktualny fork potwierdza, że natywna integracja na poziomie planu zapytania w Sailu wymaga dziś forka; oficjalny, bezforkowy mechanizm (SailExtension/FFI) jest w fazie projektowej.